Robotic Perception and Uncertainty

INF8422: Robotic Perception and Spatial Intelligence

Prof. Pierre-Yves Lajoie

POLYTECHNIQUE MONTRÉAL

Agenda

1. Motivation and Context
2. Sources of Uncertainty
3. Measuring Uncertainty
4. Types and Representation of Uncertainty
5. Quantifying Uncertainty
6. Calibrating Uncertainty
7. Applications in Robotic Perception

Motivation and Context

Understand → Measure → Quantify → Calibrate → Act

The Impact of Noise

Perception	**Planning**	**Control**
Sensors, vision, LiDAR	Trajectories, decisions	Motors, actuators
Understand the world	*Choose the actions*	*Execute the actions*

Fundamental Principle

Every sensor, without exception, is corrupted by **noise** (thermal, electrical, mechanical drift, vibrations, etc.). If poorly modeled, this uncertainty propagates **in cascade**: distorted data $\rightarrow$ suboptimal decisions $\rightarrow$ incorrect actions.

Course Objective

1. **Identify the sources** of uncertainty in a robotic system (hardware, model, environment).
2. **Classify** the types of uncertainty: aleatoric (data) vs. epistemic (model).
3. **Measure** uncertainty with appropriate metrics: variance, entropy, NLL, ECE.
4. **Quantify** uncertainty with learning methods: ensembles, MC Dropout, conformal prediction.
5. **Calibrate** a model so that its probabilities are reliable.
6. **Act** with uncertainty: compare decisions, plan cautiously, and explore to reduce what the robot doesn't know.

What Is Uncertainty?

Before quantifying uncertainty, we need to agree on what a **probability** is. Two interpretations coexist, and both will be used in this course.

Frequentist School

Probability is a **limiting frequency** over repeated experiments:

$$P(A) = \lim_{n \to \infty} \frac{n_A}{n}$$

- The parameter θ is **fixed but unknown**; it is the **data** that is random.

Bayesian School

Probability is a **degree of belief** conditioned on the available information:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta)$$

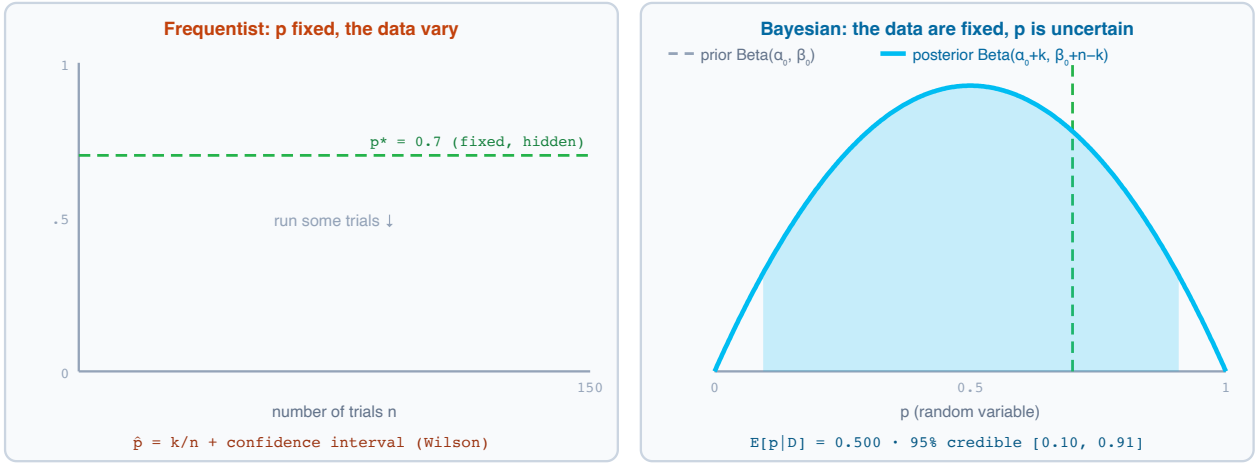
- The parameter θ is a **random variable**; we condition on the **observed** (fixed) data.

Why It Matters Here

"The uncertainty" of a prediction does not mean the same thing in the two schools: **variability under repetition** (frequentist) vs. **belief about what we don't know** (Bayesian).

Frequentist vs. Bayesian — Interactive Visualization

The **same stream of Bernoulli trials** (true success rate $p^* = 0.7$, hidden) read by the two schools: on the left $\hat{p} = k/n$ fluctuates around a fixed p^*; on the right the belief about p concentrates.



+1 trial | +10 trials | Reset | prior strength $\alpha_0=\beta_0$ 2 | n = 0, k = 0 successes

Uncertainty in Sensor Fusion

A robot generally has **several sensors** (camera, LiDAR, GPS, IMU…). Faced with conflicting measurements, uncertainty serves as a **confidence weight**.

Bayesian Fusion

Given two independent sensors providing measurements z_1 and z_2 about the state $\mathbf{x}$:

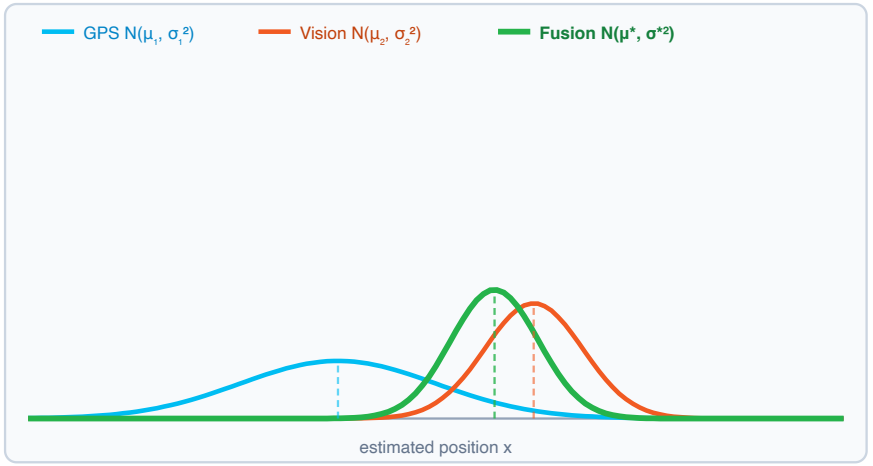
$$p(\mathbf{x} \mid z_1, z_2) \propto \underbrace{p(z_1 \mid \mathbf{x})}_{\text{sensor 1 model}} \cdot \underbrace{p(z_2 \mid \mathbf{x})}_{\text{sensor 2 model}} \cdot \underbrace{p(\mathbf{x})}_{\text{prior}}$$

The **more accurate** sensor (lower variance) contributes more to the final estimate.

Intuition: If GPS is uncertain (poor satellite visibility) but vision is confident (clear visual landmarks), the fusion naturally weights in favor of vision.

Sensor Fusion — Interactive Visualization

The product of two Gaussians is a **narrower** Gaussian: adjust the precisions and observe which one dominates the fusion.



Bayesian fusion

$$1/\sigma^{*2} = 1/\sigma_1^2 + 1/\sigma_2^2$$

$$\mu^* = \sigma^{*2}(\mu_1/\sigma_1^2 + \mu_2/\sigma_2^2)$$

$\sigma_1 = 1.20$ $(\mu_1 = 3.8)$

$\sigma_2 = 0.60$ $(\mu_2 = 6.2)$

$\sigma^* = 0.537$

$\mu^* = 5.72$

$\sigma^* < \min(\sigma_1, \sigma_2) = 0.60$ ✓
Fusion always improves precision

μ^* leans toward the more precise sensor

σ_1 — GPS **1.20**

σ_2 — Vision **0.60**

Deciding and Predicting Under Uncertainty

Uncertainty about the state of the world **grows with the time horizon**.

Deciding: Mars rover

- **Probabilistic** elevation map: a *distribution* of maps, not a single one.
- Allows trading off risk of falling vs. energy consumption → **risk-aware** decision.
- Without uncertainty: a single decision.

Predicting: pedestrian tracking

- From t_0 to t_1, the uncertainty "bubble" **propagates and grows**.
- Tools: scenario sampling, POMDPs, and risk-sensitive planning.
- Mitigating actions: get closer, ask for help, cautious trajectory.

The Problem with *Point Estimates*

A Problem with Neural Networks

These networks are trained to produce a **single deterministic output**: the point estimate θ^* of the MLE, with no distribution over θ. They are often **overconfident**: they don't know that they don't know.

Concrete consequences:

- An autonomous vehicle classifying a white trailer as "sky" with confidence $\longrightarrow$ collision.
- A network 99% confident on an *adversarial* image (noise imperceptible to the human eye).
- A pedestrian detector unaware it is looking at an object never seen during training.
- A neural SLAM system that *silently* loses its localization with no alarm.

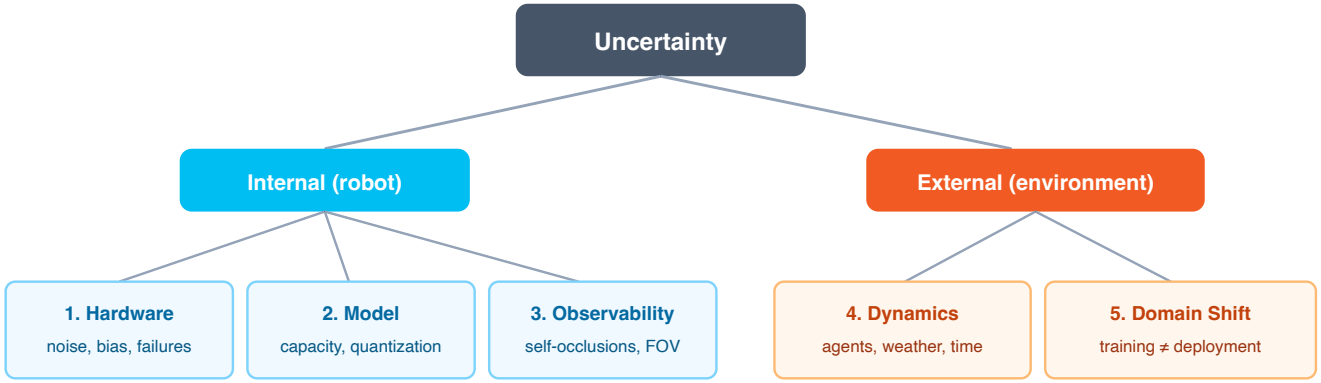
Goal: Reconnect with probabilism and force networks to quantify their ignorance.

Sources of Uncertainty

Understand → Measure → Quantify → Calibrate → Act

Where Does Uncertainty Come From?

Uncertainty in robotics comes from **internal** sources (related to the robot) and **external** sources (related to the environment).



Source 1: Hardware Limitations

All physical measurement devices exhibit **unavoidable errors**.

Types of Measurement Errors

- **Systematic errors**: Constant bias (e.g., calibration offset of a camera).
- **Random errors**: Thermal, electronic noise (e.g., Gaussian noise on a depth sensor).

Compound example: An autonomous vehicle attempts to localize a pedestrian.

- The **LiDAR** introduces a distance error (± 2 cm).
- The **GPS** is imprecise in urban environments (multi-path, ± 5 m).
- The two uncertainties **compound**: the total uncertainty on the pedestrian's position is much greater than each individual error.

Source 2: Partial Observability

A robot never perceives the world in its **entirety**.

- **Occlusions**: An object hidden behind another (pedestrian behind a car).
- **Visual conditions**: Fog, darkness, sun glare.
- **Clutter**: Dense scenes with many overlapping objects.
- **Limited field of view**: A sensor sees only a portion of the space.

Mitigation: Use multiple sensors (complementary modalities), plan movements to reduce occlusions, maintain a temporal memory.

Source 3: Environment Dynamics

The world in which the robot operates **changes over time**.

Dynamic Elements

- **Mobile agents**: Pedestrians, cyclists, other vehicles.
- **Displaced objects**: Furniture in a house, objects on a table.
- **Weather conditions**: Rain, wind, changing lighting.

Consequences

- The map built at t_0 is **stale** at t_1.
- Trajectory predictions are **uncertain**.
- Control (e.g., a drone) disturbed by wind → imprecise commands.

Growing Uncertainty Over Time

The further the prediction horizon, the greater the uncertainty about the future state. A robot planning 10 seconds ahead is far more uncertain than one planning 1 second ahead.

Source 4: Domain Shift

A model trained on one dataset can **fail** when facing situations it was never exposed to.

> **Definition**
>
> **Domain shift** occurs when the distribution of the test data $p_{test}(\mathbf{x})$ differs from the training distribution $p_{train}(\mathbf{x})$. The model encounters inputs for which it has **no experience**.

Examples:

- Autonomous driving system trained in Arizona → deployed in Montreal in winter.
- Urban segmentation model → deployed in an unfamiliar **rural** area.

Concrete Examples: Three Robotic Domains

Domestic Navigation

- Semi-permanent furniture.
- Dynamic people.
- VLMs/LLMs to understand human instructions.

Manipulation

- Occluded or transparent objects.
- Unknown mass and friction.
- Uncertain deformability.
- Uncertain grasp pose.

Field Robots

- SLAM without GPS (mines, underwater, space).
- Turbulence (drones, AUVs).
- Unstructured environments.
- Dynamic elements (people, animals, other robots).

Uncertainty in SLAM

In previous courses, SLAM was formulated as **probabilistic estimation**: MAP gives the trajectory, and the information matrix $\Lambda = J^\top \Omega J$ in principle encodes all the uncertainty ($\Sigma = \Lambda^{-1}$).

> In practice, most deployed SLAM systems only exploit the **mean** (the MAP estimate) and ignore the covariance. The robot navigates as if its pose and map were perfect.

Why this pitfall exists:

- Recovering Σ requires **solving the system**: inverting (or marginalizing) Λ destroys the sparsity that makes SLAM real-time – expensive in both computation and storage on graphs with millions of variables.
- Standard benchmarks (ATE, RPE) only score the accuracy of the mean, not the quality of the uncertainty.

Consequence: localization that drifts *silently*, with no alarm. The methods in this course aim precisely to give this **introspection** back to perception systems, classical as well as neural.

Measuring Uncertainty

Understand → **Measure** → Quantify → Calibrate → Act

Uncertainty Metrics

Quantifying uncertainty is useless if we cannot evaluate and compare it.

Three Distinct Questions

1. **Dispersion:** How spread out is the prediction? (Variance, covariance, entropy)
2. **Predictive quality:** Is the predictive distribution correct relative to reality? (NLL, Brier)
3. **Distance between distributions:** Are two distributions similar? (KL, Hellinger, Wasserstein)

Practical uses in robotics:

- Comparing two uncertainty quantification methods (e.g., Ensembles vs. MC Dropout).
- Evaluating whether a model is well calibrated on a validation set.
- Detecting out-of-distribution (OOD) data by thresholding a metric.
- Guiding active exploration toward zones of maximum uncertainty.

Variance

Variance measures the *dispersion of a random variable* around its mean.

For a discrete variable X with N outcomes $\{x_1, \ldots, x_N\}$ and probabilities $\{p_1, \ldots, p_N\}$:

1. Mean: $\mu = \mathbb{E}[X] = \sum_{n=1}^{N} p_n x_n$
2. Variance:

$$\sigma^2 = \mathbb{E}\left[(X - \mu)^2\right] = \sum_{n=1}^{N} p_n (x_n - \mu)^2$$

Important properties:

- $\sigma^2 \geq 0$ (always non-negative). $\sigma^2 = 0$ iff X is deterministic.
- The **standard deviation** $\sigma = \sqrt{\sigma^2}$ is in the same units as X (more interpretable).
- For a continuous distribution $p(x)$: $\sigma^2 = \int (x - \mu)^2 p(x)\, \mathrm{d}x$.

Limitation: Only measures the dispersion of a *single* variable (→ **covariance**, next slide). Does not capture the shape of the distribution (a bimodal distribution can have the same variance as a unimodal one).

Covariance

Two variables of the same state (e.g., x and y of a pose) do not vary independently.

Covariance measures their *co-variation*:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$$

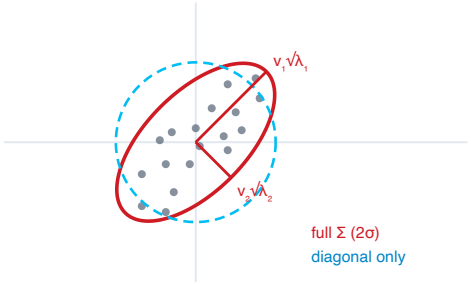
- Sign = direction of co-variation; normalized version: **correlation** $\rho = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.

Covariance matrix for $\mathbf{x} \in \mathbb{R}^d$:

$$\Sigma = \mathbb{E}\left[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\right], \qquad \Sigma_{ii} = \sigma_i^2, \quad \Sigma_{ij} = \mathrm{Cov}(x_i, x_j)$$

- Symmetric **positive semi-definite**.
- Geometry: the eigenvectors of Σ give the axes of the **uncertainty ellipse**, the $\sqrt{\lambda_i}$ its half-axes.

Why it is central in robotics: Σ describes not only the magnitude of the error but also its preferred directions – for example in a pose or a SLAM map.



If Correlations Are Ignored

Keeping only the diagonal (blue dashed line) **underestimates** the uncertainty in the oblique direction.

Shannon Entropy

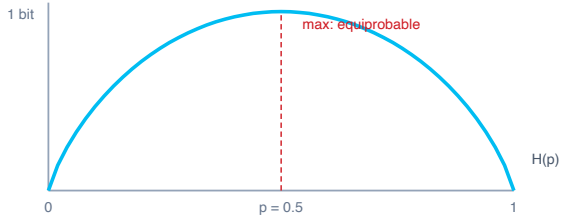
Entropy measures how random the possible outcomes of a system are. Unlike variance, it accounts for the **shape** of the distribution, not the numerical values.

For a variable X with N discrete outcomes and probabilities $\{p_1, \ldots, p_N\}$:

$$H(X) = -\sum_{n=1}^{N} p_n \log p_n \qquad \text{(in bits if log base 2, in nats if natural log)}$$

Properties:

- $H(X) \geq 0$. $H(X) = 0$ iff a single outcome is certain ($p_k = 1$).
- $H(X)$ is maximal, $= \log N$, when all outcomes are equiprobable.
- For a binary classifier (p = probability of the positive class): $H = -(p \log p + (1-p) \log(1-p))$.



Negative Log-Likelihood (NLL)

The **NLL** measures the *quality of a predictive distribution relative to ground truth*. It is the metric of choice for evaluating probabilistic models.

In classification: For the true class y_t with predicted probability $\hat{p}(y_t)$:

$$\mathrm{NLL}_{\mathrm{class}} = -\log \hat{p}(y_t)$$

If the model is confident and correct ($\hat{p}(y_t) \approx 1$): $\mathrm{NLL} \approx 0$. Otherwise NLL increases.

In Gaussian regression: For a prediction $\hat{\mu}$ with variance $\hat{\sigma}^2$ and ground truth y_t:

$$\mathrm{NLL}_{\mathrm{reg}} = \frac{1}{2}\log(2\pi\hat{\sigma}^2) + \frac{(y_t - \hat{\mu})^2}{2\hat{\sigma}^2}$$

> Its expectation is minimized **if and only if** the predictive distribution $\hat{p}$ is exactly the true distribution p.

Mahalanobis Distance

The **Mahalanobis distance** measures the *distance between a point and a multivariate distribution*, taking correlations into account.

For a point $\mathbf{x} \in \mathbb{R}^d$ relative to a distribution $(\boldsymbol{\mu}, \Sigma)$:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Special cases:

- $d = 1$: $D_M = |x - \mu|/\sigma$ (distance in number of standard deviations).
- $\Sigma = I$ (identity): $D_M = \|\mathbf{x} - \boldsymbol{\mu}\|_2$ (Euclidean distance).

Applications in robotics:

- **OOD detection:** If $D_M(\mathbf{x}_{\text{test}})$ is large in feature space, the input is probably out-of-distribution.
- **Regression evaluation:** Distance between the Gaussian prediction and ground truth.
- **Sensor fusion:** Mahalanobis distance between two estimates for data association

Comparing Distributions

Third family of metrics: measuring the **gap between two distributions** p and q (posterior vs. approximation, train vs. test features, disagreement between models).

Measure	Formula	Properties	Typical use
Kullback-Leibler	$\mathbb{KL}[p \,\|\, q] = \sum_n p_n \log \frac{p_n}{q_n}$	≥ 0, zero iff $p = q$, **not symmetric**	Variational Inference
Jensen-Shannon	$\frac{1}{2}\mathbb{KL}[p \,\|\, m] + \frac{1}{2}\mathbb{KL}[q \,\|\, m]$, $m = \frac{p+q}{2}$	symmetrized KL, bounded	disagreement between ensemble members
Wasserstein *Earth Mover Distance* (W_2)	minimal cost of **optimal transport** from p to q	sensitive to geometry; slow to compute	distance between histograms, between shapes in images

Types and Representation of Uncertainty

Understand → **Measure** → Quantify → Calibrate → Act

The Two Major Types of Uncertainty

Predictive uncertainty divides into two fundamental categories with very different properties.

Aleatoric Uncertainty

Inherent to noise in the data

Part of this uncertainty is *irreducible*.

Epistemic Uncertainty

Inherent to the model

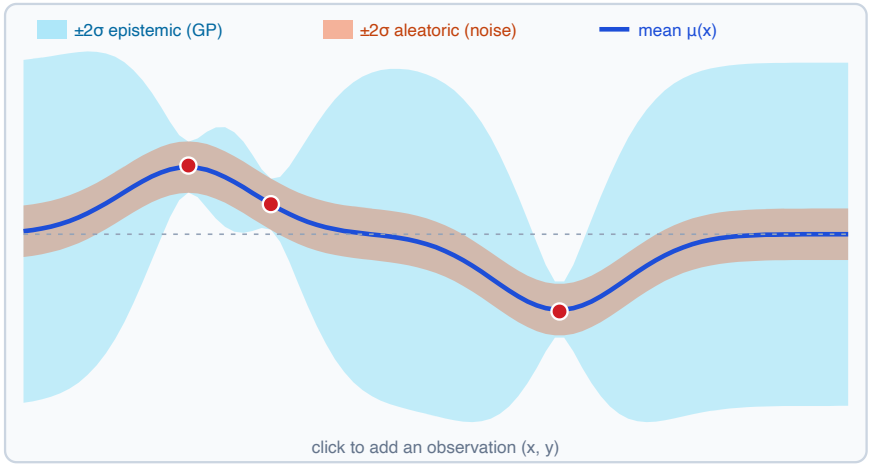
Lack of knowledge, lack of data

Limited representation unable to encode the full complexity of the data

Reducible by adding data, expanding the model, etc.

Types of Uncertainty – Interactive Visualization

Gaussian process regression: click to add observations – the **epistemic** band collapses locally, the **aleatoric** band does not move.



Two uncertainties

Aleatoric — irreducible
Constant orange band: adding points does not reduce it (σ_n = sensor noise).

Epistemic — reducible
Blue band: wide far from observations, collapses where data is available.

```
n = 3 observations
σ_alea = 0.15
ℓ = 0.80 (length scale)
```

σ_aleatoric **0.15** ℓ correlation **0.80** Reset

Aleatoric Uncertainty

Definition

Aleatoric uncertainty (also called *statistical uncertainty* or *data uncertainty*) is due to the **inherent stochasticity** of the observed phenomenon. It is **irreducible**: even with an infinite amount of data, it does not disappear.

Measurement model: For an output y given an input $\mathbf{x}$ and the true parameters θ^*:

$$y = f_{\theta^*}(\mathbf{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2_{\text{alea}})$$

where ε is the irreducible measurement noise, σ^2_{alea} its variance.

Examples in robotic perception:

- Thermal noise of a depth sensor (present even if it isn't moving).
- Semantic labeling ambiguity at the boundary between two objects.
- Motion blur in an image captured during fast movement.

Epistemic Uncertainty

Definition

Epistemic uncertainty (also called *model uncertainty* or *systematic uncertainty*) comes from a **lack of knowledge** or data. It is **reducible**: by collecting more relevant data, it decreases.

Measurement model: We seek the parameters θ of the model. Epistemic uncertainty is represented by the **posterior distribution**:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta)$$

A concentrated distribution $p(\theta|\mathcal{D})$ = low epistemic uncertainty (lots of data). A diffuse distribution = high epistemic uncertainty (little data).

Examples in robotic perception:

- Detector never trained on rare objects → high uncertainty.
- SLAM map of a never-visited area → high geometric uncertainty.
- Segmentation model facing an unknown weather condition.

Representations of Uncertainty

Uncertainty can be **represented** in different forms depending on the context.

Continuous Representations

- **Parametric distributions**: Gaussian $\mathcal{N}(\mu, \sigma^2)$, Dirichlet, von Mises-Fisher.
- **Samples** (Monte Carlo): A set of K predictions represents the posterior.
- **Density functions** (normalizing flows, GP).

Discrete Representations

- **Intervals / sets**: $[y_{low}, y_{high}]$ (conformal prediction).
- **Class sets**: {cat, tiger} (conformal classification).
- **Polytopes** and **probability boxes** (p-box) for engineering.

Practical Choice

The choice of representation depends on the **expressiveness / computational cost** trade-off. Gaussians are simple and efficient; non-parametric distributions (e.g., ensembles) are more expressive but more costly.

Homoscedastic vs. Heteroscedastic Uncertainty

Aleatoric uncertainty itself can be **constant** or **input-dependent**.

Homoscedastic

Constant variance: $\sigma^2_{\text{alea}} = \sigma^2$ (independent of $\mathbf{x}$).

- Thermal noise of a sensor.
- Uniform quantization error.

Simple to model: a single scalar to learn.

Heteroscedastic

Input-dependent variance: $\sigma^2_{\text{alea}}(\mathbf{x})$.

- Stereo depth error: $\sigma_Z \propto Z^2$ (grows with depth Z).
- Segmentation: higher uncertainty at object boundaries.

The network must **predict** the variance.

Representing Aleatoric Uncertainty: Classification

In classification with K classes, the network naturally produces a probability distribution.

The **Softmax** layer transforms the logits z_k into probabilities:

$$p(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_{k=1}^{K} p(y = k|\mathbf{x}) = 1$$

Aleatoric uncertainty measure: The entropy of the softmax distribution:

$$H = -\sum_{k=1}^{K} p_k \log p_k$$

- $H = 0$: a single class at 100% probability → very certain.
- $H = \log K$: uniform distribution → maximally uncertain.

> **Warning**
>
> This entropy is the uncertainty *as seen by a single model* θ^*: structurally **blind to the epistemic** (it can be ≈ 0 in an OOD zone).

Representing Aleatoric Uncertainty: Regression

In regression, an *uncertainty head* does not predict an abstract "confidence": it turns the network into a model of a **conditional distribution** $p(y \mid \mathbf{x})$.

Two-Head Architecture

A shared backbone produces $\mathbf{h} = f_\phi(\mathbf{x})$, then two heads predict:

$$\hat{\mu}(\mathbf{x}) = g_\mu(\mathbf{h}), \qquad \hat{\sigma}^2(\mathbf{x}) = g_\sigma(\mathbf{h}) > 0$$

- $\hat{\mu}$: the expected central value;
- $\hat{\sigma}^2$: the expected error scale for **this input**.

The variance head learns **heteroscedastic aleatoric** uncertainty. It does not by itself detect an out-of-distribution input: an epistemic method must be added to it.

Multivariate output: For $y \in \mathbb{R}^d$ (e.g., a 3D box), predict a covariance $\Sigma(\mathbf{x})$. It must remain positive semi-definite.

Cholesky trick: Predict the lower triangular matrix L such that $\Sigma = LL^\top$:

$$\hat{\Sigma} = \hat{L}\hat{L}^\top, \qquad \mathrm{diag}(\hat{L}) > 0$$

This guarantees a valid covariance while learning correlations between outputs.

Decomposition of the Total Variance

The total variance of a prediction decomposes into two independent contributions.

Total Variance

For a model with parameters θ:

$$\underbrace{\mathrm{Var}(y \mid \mathbf{x})}_{\text{total variance}} = \underbrace{\mathbb{E}_\theta\left[\sigma^2_{\text{alea}}(\mathbf{x}, \theta)\right]}_{\text{aleatoric variance}} + \underbrace{\mathrm{Var}_\theta[\mu_\theta(\mathbf{x})]}_{\text{epistemic variance}}$$

Practical interpretation:

- **Epistemic variance** = variance of the means predicted by different plausible models.
- **Aleatoric variance** = average noise predicted by these models.

This decomposition is exploited by **deep ensembles** and **MC Dropout** to separate the two types.

When the Two Types Mix

In practice, the strict aleatoric/epistemic separation can be **blurry**.

Ambiguous Examples

- **Occlusion:** An object is hidden. Is this aleatoric (physical: the object is there but invisible) or epistemic (the model never learned to infer behind occlusions of this type)?
- **Semantic boundary:** A pixel between "road" and "sidewalk". Aleatoric (inherent ambiguity) or epistemic (the model doesn't have enough boundary examples)?

Theoretical limitations:

- Dropout-type methods sometimes capture aleatoric instead of epistemic uncertainty.
- Networks dedicated to variance ($\hat{\sigma}^2$) can absorb epistemic uncertainty.
- The decomposition is imperfect, but it is a useful abstraction.

Summary: Comparison Table

Type	Cause	Reducible?	Useful representation
Aleatoric	Sensor noise, inherent stochasticity, data ambiguity	**No**	Distribution $p(y \mid x)$, variance or covariance
Epistemic	Lack of data, unseen zones, domain shift	**Yes** (more data)	Distribution over models, predictive disagreement

Key Message

For a **safe** robotic system: both types must be captured. Aleatoric to faithfully model sensor noise, epistemic to detect unknown and potentially dangerous situations.

Quantifying Uncertainty

Understand → Measure → **Quantify** → Calibrate → Act

Classical Systems Already Quantify Uncertainty

Three pillars of metrology in robotics:

📄 Datasheet

The manufacturer measures the noise **at the factory** and publishes it:

- LiDAR: ±3 cm (1σ) at 100 m.
- ToF camera: ~1% of range.
- IMU: noise density (°/s/√Hz), *bias instability*.

Always with **validity conditions**: range, temperature, target reflectivity.

🎯 Calibration

Remove the **systematic bias** and measure the residual noise:

- Checkerboard target → reprojection RMS → σ_{pixel}.
- Calibration (certificate, traceability).

🧪 Characterization (testing)

Repeated measurements under controlled conditions → **fit the noise model**

- Error histogram → Gaussian; $\sigma(Z) \propto Z^2$ in stereo (already seen).

> **There's Even a Standard for This**
>
> The **GUM** (*Guide to the Expression of Uncertainty in Measurement*, ISO) codifies this whole process.

Why the Datasheet Is Not Enough

This metrology pipeline relies on one assumption: sensor noise is **stationary** and its physics is **known**.

Unfortunately, we rarely operate under such conditions because sensor behavior also depends on the environment.

	Classical sensor	**Neural network**
Error source	known physics (thermal noise, quantization)	depends on the input **and** training data
Stationarity	stable within specified conditions	stable within the training domain
Characterization	in a lab / at the factory	must be redone for each distribution encountered

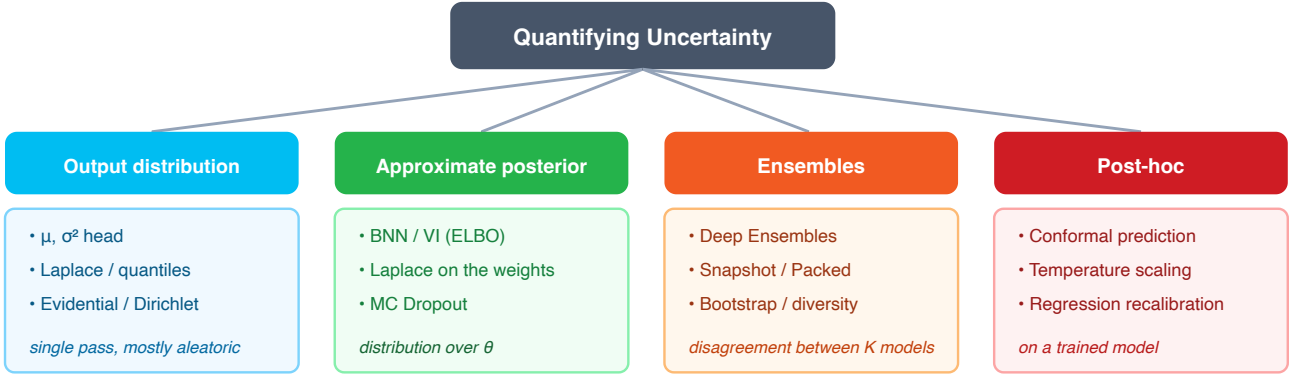
Average performance on a test set is a first source of variance estimation, but it says nothing about *this* image, *this* scene.

We will not dwell on estimating the nominal uncertainty of sensors (datasheets); see a metrology course for more details (e.g., MEC8552).

The focus will be on the uncertainty produced by perception and AI systems, and then on using it to decide and plan.

Overview: Taxonomy of Methods

How do we concretely produce a predictive distribution or ensemble? Four **complementary** families:



Combinable: a heteroscedastic head can equip each member of an ensemble.

Learning Heteroscedasticity: NLL Loss

We generally have no "true variance" target. The same labels y_i indirectly train **both heads** by maximizing their likelihood.

Gaussian Negative Log-Likelihood

Assuming $p(y_i|\mathbf{x}_i) = \mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i^2)$, the NLL loss is:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\frac{\|y_i - \hat{\mu}_i\|^2}{2\hat{\sigma}_i^2}}_{\text{precision term}} + \underbrace{\frac{1}{2}\log \hat{\sigma}_i^2}_{\text{regularization}}$$

Loss balance:

- If $\hat{\sigma}_i^2$ is too small, a mean error is very costly.
- If $\hat{\sigma}_i^2$ is too large, the term $\log \hat{\sigma}_i^2$ penalizes it.
- The optimum thus associates a small variance with predictable cases and a large variance with intrinsically ambiguous cases.

Practical Pitfall

The variance can temporarily "explain away" bad predictions by inflating. A decreasing NLL therefore doesn't necessarily mean the mean is improving: both must be monitored separately.

Which Distribution Should Be Predicted?

Adding a head first means choosing a family for $p(y \mid \mathbf{x})$. This assumption determines the network's outputs and the loss.

Probabilistic head	Outputs	Loss	When to use it
Gaussian	$\mu,\ \log \sigma^2$	Gaussian NLL	Symmetric errors, few extreme values
Laplace	median m, $\log b$	$\lvert y - m \rvert / b + \log b$	Heavier-tailed errors, robust target
Quantiles	$q_{\tau_1}, \ldots, q_{\tau_K}$	*Pinball loss*	Direct intervals, without a full parametric form
Multivariate Gaussian	$\boldsymbol{\mu},\ L$	Multivariate NLL	Correlated outputs: pose, 3D box, optical flow

Choice Principle

The head should reflect the **shape of the residuals** and the structure of the output. A Gaussian variance is useful only if this approximation is reasonable; it does not automatically represent all the uncertainty.

Ref.: Kendall & Gal, 2017.

Training a Heteroscedastic Head

Minibatch pseudo-code

```
h       ← backbone(x)
mu      ← mean_head(h)
s_raw   ← scale_head(h)
s       ← clip(s_raw, s_min, s_max)   # log variance

sq_err  ← (y - mu)²
loss    ← mean(0.5 · exp(-s) · sq_err + 0.5 · s)

zero_grad()
backward(loss)
optimizer.step()
```

What is actually learned

- $s = \log \sigma^2$ guarantees $\sigma^2 = e^s > 0$.
- e^{-s} weights each residual by the predicted precision.
- The gradient flows through both heads **and** the shared backbone.
- No variance label is required: the NLL compares the predicted distribution to the single label y.

Stable alternative: predict $r \in \mathbb{R}$, then set $\sigma^2 = \mathrm{softplus}(r) + \varepsilon$.

At inference: $\hat{\sigma} = e^{s/2}$ and, under the Gaussian assumption, an approximate 95% interval is $\hat{\mu} \pm 1.96\hat{\sigma}$.

Stabilizing and Verifying Training

Training Recipe

1. First train the mean with MSE or Huber, with a fixed variance.
2. Activate the scale head and jointly optimize the NLL.
3. If the mean stops improving, use β-NLL:

$$\mathcal{L}_\beta = \mathbf{stopgrad}\left[(\hat{\sigma}^2)^\beta\right] \mathcal{L}_{\mathrm{NLL}}, \qquad 0 \leq \beta \leq 1$$

The detached weighting changes the importance of examples without offering the variance head a new shortcut.

Verify on Validation

- **Central value:** MAE or RMSE of $\hat{\mu}$.
- **Full distribution:** NLL.
- **Intervals:** empirical coverage **and** average width.
- **Subgroups:** repeat by distance, lighting, weather, and domain.

Limitation to Remember

This head mostly learns the aleatoric uncertainty **within the training domain**. To flag the unknown, combine it with an ensemble, MC Dropout, or another epistemic method, then calibrate it on separate data.

Ref.: Seitzer et al., 2022.

Bayes' Theorem: Foundation

Most uncertainty quantification methods are grounded in **Bayes' theorem**.

For the parameters θ of a model and a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ (inputs x, outputs y):

$$\underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} = \frac{\underbrace{p(\mathcal{D} \mid \theta)}_{\text{likelihood}} \cdot \underbrace{p(\theta)}_{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{evidence}}}$$

Each term:

- $p(\theta)$: **prior** belief about the parameters (before seeing the data).
- $p(\mathcal{D}|\theta)$: **likelihood** of the data given the parameters.
- $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)\, p(\theta)\, \mathrm{d}\theta$: **evidence** (generally **intractable**, used for normalization only).
- $p(\theta|\mathcal{D})$: **posterior**, our updated belief after observing $\mathcal{D}$.

Bayesian Predictive Distribution

The final goal is to predict for a new input $\mathbf{x}^*$.

Predictive Distribution

By marginalizing over the parameter posterior:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int \underbrace{p(y^* \mid \mathbf{x}^*, \theta)}_{\text{predictive likelihood}} \cdot \underbrace{p(\theta \mid \mathcal{D})}_{\text{posterior}} \, \mathrm{d}\theta$$

This integral captures both the aleatoric ($p(y^*|\mathbf{x}^*, \theta)$) and the epistemic ($p(\theta|\mathcal{D})$).

Sampling approximation: If we have K samples $\{\theta_k\}_{k=1}^K \sim p(\theta|\mathcal{D})$:

$$p(y^* \mid \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{K} \sum_{k=1}^{K} p(y^* \mid \mathbf{x}^*, \theta_k)$$

The problem is obtaining these samples θ_k from the posterior, which is what the various methods do.

Bayesian Neural Networks (BNN)

Core Idea

Instead of a single weight w_{ij} per connection, place a **probability distribution** $q(w_{ij})$ over each weight. The network thus represents a distribution over possible networks.

For a network with weights $\mathbf{w}$, we learn a variational distribution $q(\mathbf{w}) = \prod_{ij} \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$.

Architectural consequences:

- Each weight w_{ij} is replaced by **two parameters** $(\mu_{ij}, \sigma_{ij}^2)$ → doubling memory.
- At inference, we **sample** $\mathbf{w} \sim q(\mathbf{w})$ and perform a classical forward pass.
- The predictive distribution is obtained by repeating this process K times.

Advantage: Mathematically rigorous framework for epistemic uncertainty.

Disadvantages: $\times 2$ memory, numerical instability, training difficulties.

Variational Inference (VI) and ELBO

How do we train a BNN if the posterior $p(\mathbf{w}|\mathcal{D})$ is intractable? By using **Variational Inference**.

Goal: Find $q(\mathbf{w})$ within a parametric family that *best approximates* $p(\mathbf{w}|\mathcal{D})$, by minimizing $\mathbb{KL}[q(\mathbf{w})||p(\mathbf{w}|\mathcal{D})]$.

Since this KL cannot be computed directly, we maximize the **Evidence Lower BOund** (ELBO):

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(\mathbf{w})}[\log p(\mathcal{D} \mid \mathbf{w})]}_{\text{expected likelihood}} - \underbrace{\mathbb{KL}[q(\mathbf{w})||p(\mathbf{w})]}_{\text{prior regularization}}$$

- **First term:** Rewards q for explaining the training data well (*fit*).
- **Second term:** Penalizes q for straying too far from the prior $p(\mathbf{w})$ (regularization).

Monte Carlo Dropout (MC Dropout)

Dropout, usually used only during training for regularization, can be kept active at **inference** as a Bayesian approximation.

Algorithm:

1. Keep Dropout active at inference (randomly disable neurons on each forward pass).
2. Pass the same input $\mathbf{x}$ through the network N times.
3. Compute the mean and variance of the N predictions:

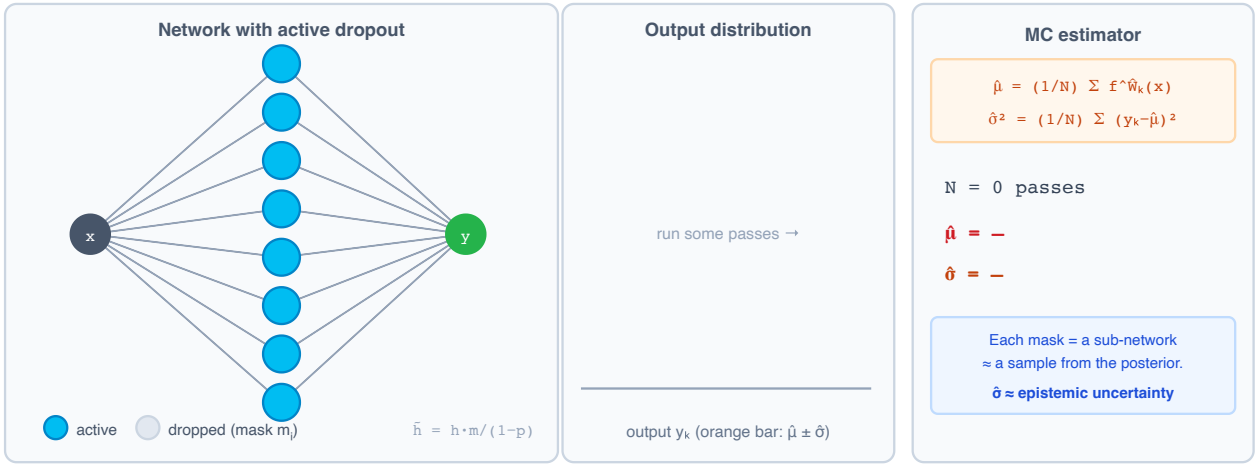
$$\hat{\mu} \approx \frac{1}{N} \sum_{i=1}^{N} f^{\hat{\mathbf{W}}_i}(\mathbf{x}), \qquad \hat{\sigma}^2_{\text{epis}} \approx \frac{1}{N} \sum_{i=1}^{N} \left(f^{\hat{\mathbf{W}}_i}(\mathbf{x}) - \hat{\mu} \right)^2$$

Advantages: Zero architectural modification, easy to implement.

Disadvantages: $\mathcal{O}(N)$ inference time, weak Bayesian approximation.

MC Dropout – Interactive Visualization

A real $1 \to 8 \to 1$ network: each pass draws a dropout **mask** and executes a real forward pass. The spread of the outputs estimates the uncertainty.



1 pass | 20 passes | Reset | dropout p **0.30**

Deep Ensembles

Principle: Train K **independent** networks with different weight initializations.

$$\hat{\mu} = \frac{1}{K} \sum_{k=1}^{K} \mu_k(\mathbf{x})$$

$$\hat{\sigma}^2 = \underbrace{\frac{1}{K} \sum_{k=1}^{K} \left(\mu_k(\mathbf{x}) - \hat{\mu}\right)^2}_{\text{epistemic}}$$

Advantages: Generally quite accurate, parallelizable.

Disadvantages: $K\times$ memory, training, and inference cost.

Deep Ensembles — Interactive Visualization

K differently trained regressors (random features + different bootstraps): they agree on the data and diverge in the OOD zone.



At test point x*

$$\sigma^{*2} = (1/K)\Sigma\sigma_k^2 + (1/K)\Sigma(\mu_k - \bar{\mu})^2$$

x* = 2.00 μ̄ = 0.14

σ²_epist = 0.270

K members **5**

Test point x* **2.0**

ID point | OOD point

Efficient Ensemble Variants

The $K\times$ cost of deep ensembles has motivated several **lighter variants**.

Snapshot Ensembles

- A single training run with a **cyclic** learning rate.
- Save the model at each local minimum ("snapshot").
- K snapshots = K ensemble members.
- Training cost: $1\times$ instead of $K\times$.

Packed Ensembles

- K sub-networks within a **single architecture**.
- Parallel computation on GPU.
- Reduced memory cost.

BatchEnsemble (Wen et al., 2020)

- Shared weights + **rank-1** factors per member: $W_k = W \odot r_k s_k^\top$.
- Near $1\times$ memory, members computed in a single batch.

Adversarial Training

- Train the members with loss functions that maximize their diversity.

Conformal Prediction: Learning an Error Margin

A model predicts $\hat{y}(\mathbf{x})$, but this value alone doesn't say **how far its error can go**.

The Idea in One Sentence

We hold out a **calibration** set, observe the model's errors on this data, then choose a margin $\hat{q}$ cautious enough to enclose new predictions.

Regression

$$\hat{y}(\mathbf{x}^*) \quad \longrightarrow \quad [\hat{y}(\mathbf{x}^*) - \hat{q},\ \hat{y}(\mathbf{x}^*) + \hat{q}]$$

The output is a **band of plausible values**.

Classification

$$\text{cat} \quad \longrightarrow \quad \{\text{cat},\ \text{tiger}\}$$

The output is a **set of plausible classes**.

Conformal prediction does not retrain the model: it **calibrates the size of the returned set** based on its past errors.

Split Conformal: Building the Band

1. Measure the Errors

On n calibration data points, compute:

$$s_i = |y_i - \hat{y}(\mathbf{x}_i)|$$

2. Choose a Cautious Margin

Sort $s_{(1)} \leq \cdots \leq s_{(n)}$, then:

$$k = \lceil (n+1)(1-\alpha) \rceil, \qquad \hat{q} = s_{(k)}$$

3. Enclose the New Prediction

$$\mathcal{C}(\mathbf{x}^*) = [\hat{y}(\mathbf{x}^*) - \hat{q},\ \hat{y}(\mathbf{x}^*) + \hat{q}]$$

Example: n = 20, target = 90%

$$k = \lceil 21 \times 0.90 \rceil = 19$$

We take the **19th smallest** calibration error. If $s_{(19)} = 0.44$, then $\hat{q} = 0.44$.

For a new prediction $\hat{y}(\mathbf{x}^*) = 0.60$:

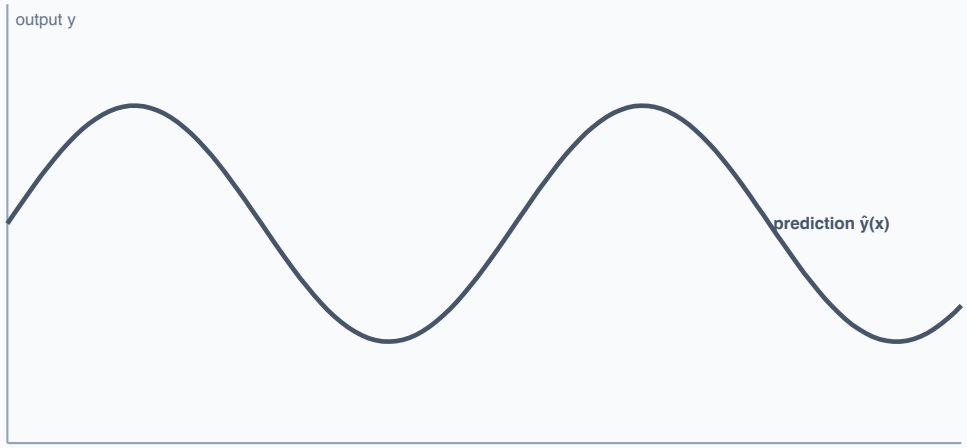
$$\mathcal{C}(\mathbf{x}^*) = [0.16,\ 1.04]$$

Interpretation: The true value lies within this interval in 90% of cases.

From Prediction to Conformal Interval

Click through the four steps: **calibration errors** become an uncertainty margin, then an interval for a new prediction.

1 **Predict** | 2 Measure errors | 3 Choose $\hat{q}$ | 4 Predict an interval



STEP 1

The model predicts a central value

$$\hat{y}(x)$$

The curve says what the model expects, but it gives **no margin of error** yet.

A prediction alone does not express its reliability.

Comparison of Methods

Method	Epist.	Aleat.	Post-hoc	Training cost	Guarantees
Deep Ensembles	✓	≈	No	$K\times$	No
MC Dropout	≈	✓	Yes	$1\times$	No
Laplace	✓	No	Yes	$1\times$	No
VI (BNN)	✓	✓	No	2–$5\times$	No
Conformal	≈	≈	Yes	$1\times$	**Yes**

≈: partial approximation.

Calibrating Uncertainty

Understand → Measure → Quantify → **Calibrate** → Act

Calibration vs. Sharpness

Two distinct, complementary properties characterize a good predictive distribution.

Calibration

Statistical consistency between predicted probabilities and observed frequencies.

Example: If the model predicts "car" with probability 0.7 on 100 images, then about 70 (on average) should indeed be cars.

A perfectly calibrated model = confidence = actual accuracy.

Sharpness

Concentration of the predictive distribution. A narrow peak = *informative* distribution.

Example: Predicting with 99% confidence is "sharper" than 60%.

A sharp **but poorly calibrated** distribution is dangerous!

The Goal

Maximize **sharpness** while guaranteeing **calibration**. Together, these two define a good predictive distribution for robotic safety.

Reliability Diagram

The **reliability diagram** is the standard visualization tool for assessing calibration.

Construction:

1. Partition the predictions into M bins based on **predicted confidence** $\hat{p}$.
2. For each bin B_m: compute the **actual accuracy** $\mathrm{acc}(B_m)$ (fraction of correct predictions).
3. Plot $\mathrm{acc}(B_m)$ vs. $\mathrm{conf}(B_m)$ (mean confidence of the bin).

Reading the graph:

- Perfect diagonal = ideal calibration.
- Above the diagonal = **underconfident**.
- Below the diagonal = **overconfident**.

Expected Calibration Error (ECE)

ECE is the most popular scalar metric for evaluating calibration.

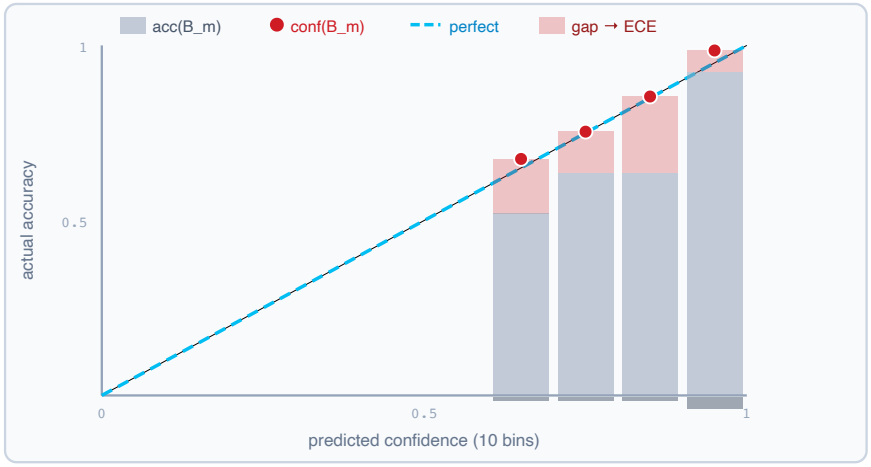
Partition the N predictions into M bins $\{B_m\}_{m=1}^{M}$ based on predicted confidence. The ECE is:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \underbrace{\frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}[\hat{y}_i = y_i]}_{\mathrm{acc}(B_m)} - \underbrace{\frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i}_{\mathrm{conf}(B_m)} \right|$$

- ECE $= 0$: perfect calibration.
- Weighted by bin size $|B_m|/N$ → bins with more samples contribute more.
- **ACE** (*Adaptive CE*): variant with equal-sized bins (the same number of samples) rather than uniform intervals → more robust to imbalanced distributions.

ECE — Interactive Visualization

The bias slider shifts the **confidences** without changing the **actual successes**: the colored gaps are each bin's contribution to the Expected Calibration Error (ECE).



Expected Calibration Error

$$ECE = \Sigma\ (|B_m|/N) \cdot |acc - conf|$$

9.6%

confidence bias: +0.15

OVER-confident

Bias shifts confidences, not actual successes: the red points leave the diagonal.

N = 1000 predictions, M = 10 bins

Confidence bias **+0.15**

Over-confident (e.g. +0.20) | Calibrated (0) | Under-confident (e.g. −0.15)

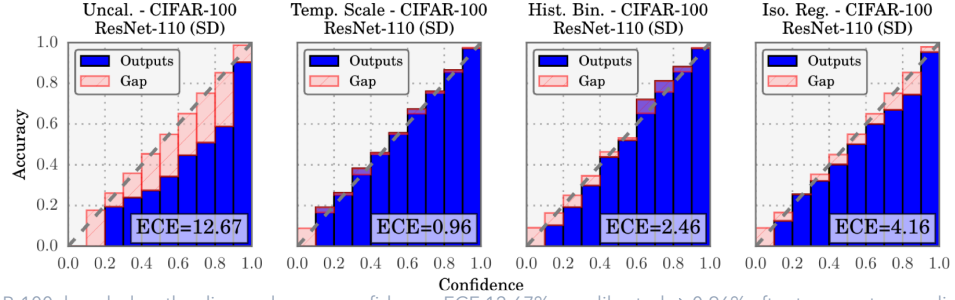
Modern Networks Are Poorly Calibrated

Empirical Observation (Guo et al., 2017)

Comparing deeper contemporary networks (ResNet, Inception) to networks from the 1990s-2000s: top-1 accuracy **increased**, but calibration **degraded**. Deep networks are **systematically overconfident**.

Likely causes:

- **Over-parameterization:** A network with millions of parameters can perfectly interpolate the training data → artificial confidence.
- **Prolonged training:** Can lead to overfitting.



ResNet-110 on CIFAR-100: bars below the diagonal = overconfidence. ECE 12.67% uncalibrated → 0.96% after temperature scaling – Guo et al., 2017.

Temperature Scaling

Temperature Scaling is the most popular post-hoc calibration method thanks to its *simplicity* and *effectiveness*.

Principle: Divide the logits $\mathbf{z}$ by a scalar $T > 0$ before the softmax:

$$\hat{q}_k = \frac{\exp(z_k/T)}{\sum_{j=1}^{K} \exp(z_j/T)}$$

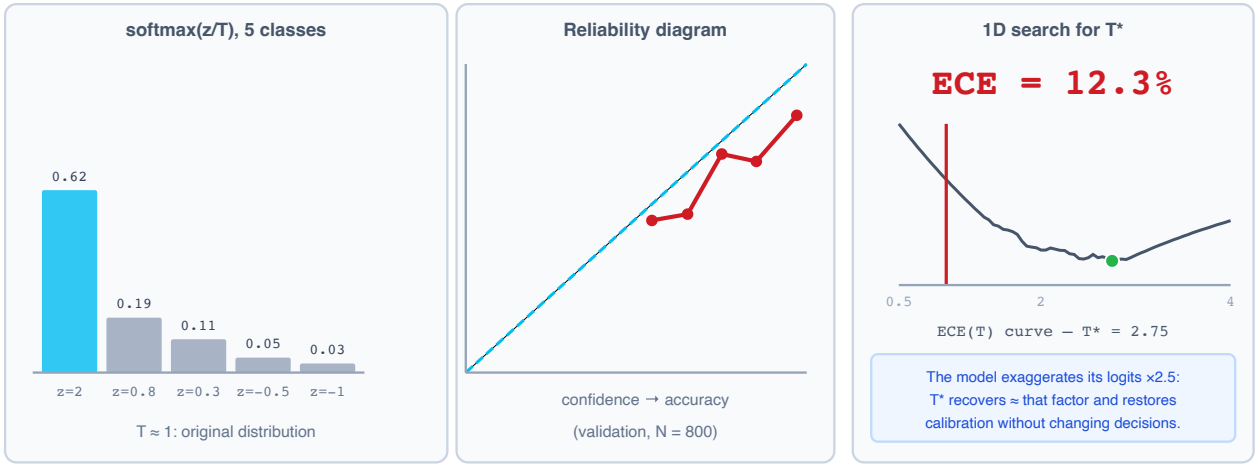
- $T = 1$: identical to the original model.
- $T > 1$: softer distribution (less confident).
- $T < 1$: sharper distribution (more confident).

Optimization: T is chosen on a **validation** set to minimize the NLL or ECE.

Advantages: A single parameter, does not change top-1 accuracy, post-hoc.

Limitations: Same T for all classes. Fails under domain shift.

Temperature Scaling – Interactive Visualization



Temperature T **1.00**

Find optimal T

T = 1

Histogram Binning

Histogram Binning is a **non-parametric** calibration method that directly adjusts probabilities bin by bin.

Algorithm:

1. Partition $[0, 1]$ into M bins $\{B_m\}_{m=1}^{M}$ (confidence intervals).
2. On the calibration set, compute the actual accuracy in each bin: $\theta_m = \mathrm{acc}(B_m)$.
3. At inference: if the model predicts a confidence $\hat{p}$ in bin B_m, return θ_m instead.

Advantages: Non-parametric, flexible, can correct complex calibration errors.

Disadvantages:

- Requires a large calibration set ($n \gg M$).
- Sensitive to the choice of number of bins M.
- Does not generalize well outside the observed bins.

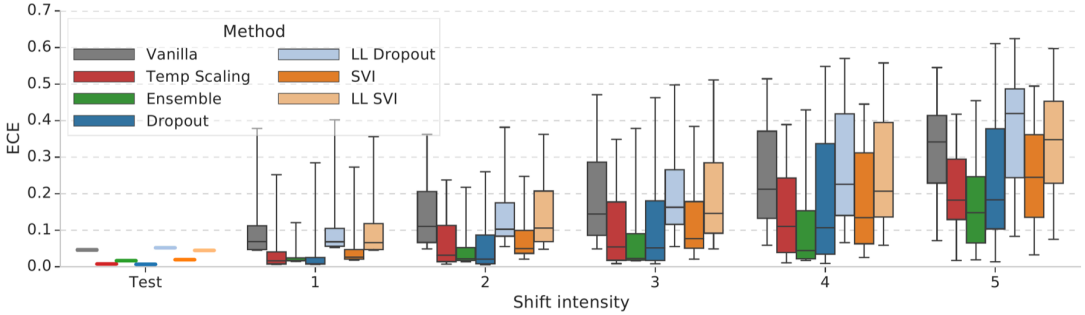
Calibration Quality Under Domain Shift

A crucial question in robotics: are calibration methods **robust** to changing conditions?

Results from Ovadia et al. (2019)

Comparative study of several methods (Temperature Scaling, MC Dropout, Ensembles, VI) under varying perturbation strengths (corruption, domain shift):

- Uncertainty quality degrades with perturbation strength for **all** methods.
- **Deep Ensembles** and **stochastic VI** degrade the slowest.
- Temperature Scaling alone is **insufficient** against domain shift.



ECE on corrupted CIFAR-10: the ECE of all methods grows with shift intensity, but the ensemble (green) stays lowest – Ovadia et al., 2019.

Summary: When and How to Calibrate?

Recommended Protocol

1. **Always evaluate** calibration on a separate validation set.
2. **Visualize** the reliability diagram to understand the direction of the bias.
3. **Compute** the ECE and NLL as metrics.
4. If ECE $> 5\%$: apply a post-hoc method.

When to Use Temperature Scaling:

- Test distribution similar to training.
- Very limited computational budget.
- Quick first baseline.

When to Prefer Other Approaches:

- Expected domain shift → Ensembles.
- Formal guarantees → Conformal.

Fundamental Principle

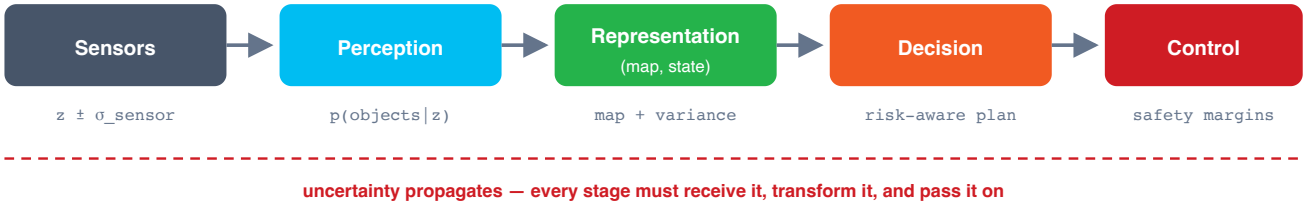
Post-hoc calibration (e.g., temperature scaling) does not replace **good uncertainty quantification** at the source. It improves an already reasonable model; it cannot fix a fundamentally deficient one.

Applications in Robotic Perception



Overview: Uncertainty in the Robotic Pipeline

Quantified uncertainty comes into play at every stage of the robotic pipeline.



Perception

- Object detection
- Segmentation
- Pose estimation

Representation

- Mapping
- Localization
- World model

Decision

- Planning
- Control
- Active exploration

Probabilistic Mapping

The **map** is the fundamental representation of the world for a robot. Representing its uncertainty is critical.

Classical Occupancy Grids

- Grid of cells: $p(\text{occupied}|z_{1:t})$.
- Recursive Bayesian filter via log-odds.

Advanced Probabilistic Maps

- **GP maps:** Estimate both aleatoric **and** epistemic uncertainty; continuous and differentiable.
- **Bayesian Hilbert Mapping (BHM):** VI for scalability over large areas.

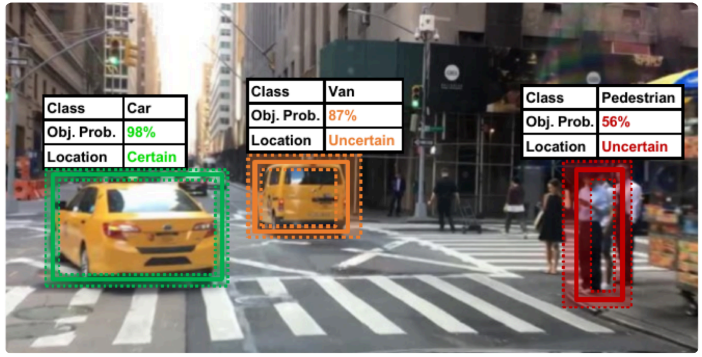
Usefulness of map uncertainty: Identify unvisited areas (high epistemic uncertainty) to guide exploration, or dynamic areas for replanning.

Probabilistic Object Detection

Scenario: an ensemble detector ($K=5$) perceives a **pedestrian** at the roadside.

Probabilistic output of the detector:

- Class: $p(\text{pedestrian}) = 0.85$ (average softmax across members).
- Distance: $\hat{\mu}_z = 12.0$ m, $\hat{\sigma}_z = 0.8$ m.
- 3D box (x, y, z, w, h, l): a $(\hat{\mu}, \hat{\sigma}^2)$ pair per parameter.



Ideal probabilistic detector: class probability and localization uncertainty (dashed boxes) per object – Feng et al., 2021.

6DoF Pose Estimation and Semantic Segmentation

Same principles, different output heads.

6DoF Pose ($\mathrm{SE}(3)$)

Estimate (x, y, z) + (roll, pitch, yaw) = 6 degrees of freedom.

- **Translation:** Multivariate Gaussian $\mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$.
- **Rotation:** On $SO(3)$, specialized distributions:
  - *von Mises-Fisher*: Distribution on the sphere.
  - *Bingham*: For quaternions (antipodal).
- **Ensembles** for 6DoF pose (Wursthorn et al., 2024).

Semantic Segmentation

Assign a class to each pixel. Two types of uncertainty:

- **Class uncertainty:** Pixel ambiguous between two categories (e.g., road/sidewalk).
- **Boundary uncertainty:** Pixels at object boundaries, critical for safety in autonomous driving.

Out-of-Distribution (OOD) Detection

A safe robotic system must know **when it is outside its domain of competence**.

> **Principle**
>
> If the epistemic uncertainty is high for an input $\mathbf{x}^*$, this is a signal that $\mathbf{x}^*$ is **out-of-distribution** (OOD): the model has not learned to handle similar inputs.

OOD detection methods:

- **Mahalanobis distance** in feature space: if $D_M(\phi(\mathbf{x}^*))$ is large, $\mathbf{x}^*$ is OOD.
- **High ensemble variance** → strong OOD signal.
- **Entropy** of the predictive distribution.

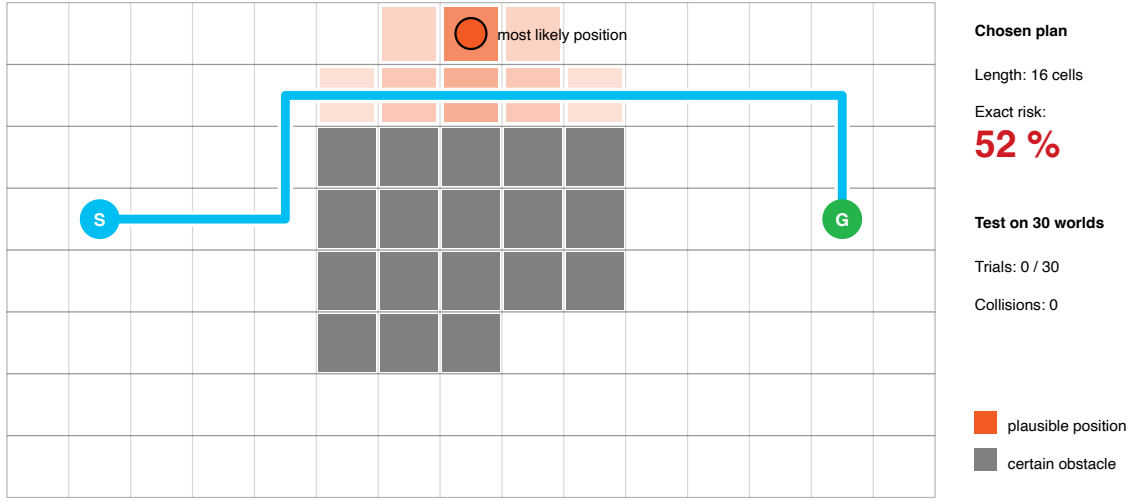
Applications:

- Autonomous driving: kangaroo detection (Nitsch et al., 2021).
- Manipulation: unknown object → ask for help.
- Space robots: pose estimation for an uncataloged object.

Ideally we want **"The model to know what it doesn't know."**

Planning Under Uncertainty – Interactive Visualization

The most probable position of an obstacle is not the whole story. Compare a short route based on this single position with a route that accounts for **all plausible positions**, then test both in 30 possible worlds.



Mean position | Safety first | Test 30 possible worlds

The route is short, but it crosses plausible positions of the object: risk 52 %.

Open Challenges and Conclusion

Despite considerable progress, several **open challenges** remain.

Scientific Challenges

- **Blurry separation:** Aleatoric vs. epistemic is not always clear, depends on the level of description.
- **Ground truth:** The "true" uncertainty of an image is unknown. Photorealistic simulators help but are imperfect.
- **Calibration under shift:** No method remains perfectly calibrated out-of-distribution.

Practical Challenges

- **Scalability:** Ensembles and BNNs are too slow, especially for drones or real-time surgery.
- **Large models:** How do we quantify uncertainty in LLMs and VLMs?
- **Standardization:** No universal protocol for evaluating uncertainty.

Final Message

Robust robotic perception is no longer measured by accuracy alone (mIoU, AP). **Introspective capacity** – *knowing what you don't know* – defines the safety and reliability of an autonomous system. Quantifying, calibrating, propagating, and then **acting with uncertainty** are fundamental skills of robotic perception.

Appendix – Distances Between Distributions

Appendix A1: Kullback-Leibler (KL) Divergence

The **KL divergence** measures the *difference between two probability distributions*. It is a central tool of Bayesian inference.

For two discrete distributions $p = \{p_n\}$ and $q = \{q_n\}$:

$$\mathbb{KL}[p||q] = \sum_{n=1}^{N} p_n \log \frac{p_n}{q_n}$$

Fundamental properties:

- $\mathbb{KL}[p||q] \geq 0$. Zero if and only if $p = q$.
- **Not symmetric**: $\mathbb{KL}[p||q] \neq \mathbb{KL}[q||p]$ in general.
- Central role in **Variational Inference**: minimizing $\mathbb{KL}[q(\theta)||p(\theta|\mathcal{D})]$.

The Jensen-Shannon divergence (JSD) is a symmetrized version:

$$\mathrm{JSD}(p, q) = \frac{1}{2}\mathbb{KL}[p||m] + \frac{1}{2}\mathbb{KL}[q||m], \quad m = \frac{p+q}{2}$$

Appendix A2: Wasserstein Distance

The **Wasserstein distance** (or optimal transport distance) measures the difference between distributions while taking the **geometry of the space** into account.

The **2-Wasserstein** (W_2) between p (carried by points $\{x_i\}$) and q (carried by $\{y_j\}$):

$$W_2^2(p, q) = \min_{M \geq 0} \sum_{i,j} M_{i,j} \|x_i - y_j\|_2^2, \qquad W_2 = \sqrt{W_2^2}$$

subject to the marginal constraints: $M\mathbf{1} = p$, $M^\top \mathbf{1} = q$ (M_{ij} = mass transported from x_i to y_j).

Intuition: W_2 is the minimal cost to "move" distribution p toward q (*Earth Mover's Distance*).

Advantages over KL:

- Defined even when the supports of p and q do not overlap.
- Geometrically sensitive: two nearby Gaussians have a small Wasserstein distance.
- Used to evaluate generative models (GANs) and in learning uncertainty measures.